



HARBORVIEW GRAND

Luxury Waterfront Hotel — AI Concierge System

Complete Project Report & Documentation

Full-Stack AI-Powered Hotel Guest Assistant

DEVELOPER

Sumit Kumar Singh

ASSIGNMENT

Full-Stack AI Hotel Guest Assistant

REPOSITORY

github.com/Sumitboii/HARBORVIEW-GRAND

LIVE DEMO

harborview-grand.onrender.com

STACK

Next.js 14 · Express.js · Google Gemini 2.0 Flash

TESTS

27 Jest · 4 Playwright E2E — 100% passing

Table of Contents

1.	Assignment Overview	3
2.	What Was Built	4
3.	Tech Stack — What, Why & How	5
4.	Project Structure	6
5.	Architecture & Data Flow	7
6.	Backend Deep Dive	8
7.	Frontend Deep Dive	9
8.	AI Integration	10
9.	Knowledge Base	11
10.	API Reference	12
11.	Test Coverage	13
12.	Assignment Completion Checklist	14
13.	Design Decisions & Trade-offs	15
14.	How to Run Locally	16

Assignment Overview

The assignment required building a full-stack AI-powered **Hotel Guest Assistant** — a chat interface where hotel guests can interact naturally with hotel services.

Core capabilities required:

- Ask natural-language questions about hotel policies, amenities, rooms, and services
- Check live room availability using a structured date/guest form
- Hold multi-turn conversations with context carried across messages
- Receive grounded, hallucination-free answers from a hotel knowledge base

Required Questions to Handle

#	Question	Type
1	What time is check-in?	FAQ / Policy
2	Does the hotel have a swimming pool?	Amenity
3	Which room is suitable for three guests?	Room Info
4	Is breakfast included?	Policy
5	What is the cancellation policy?	Policy
6	Do you have rooms available for a given date?	Availability

All six are fully implemented and tested end-to-end.

What Was Built

Harborview Grand AI Concierge — a two-panel luxury hotel assistant with a left sidebar showing live hotel info and shortcuts, and a right chat panel powered by Google Gemini.

Key Features Delivered

- **Natural language Q&A** — Guests type any question; the system retrieves relevant hotel knowledge and generates a natural response via Gemini.
- **Room availability checker** — Structured form for check-in/check-out dates and guest count; returns real-time per-room inventory.
- **Multi-turn conversation** — Conversation ID maintained across messages; follow-up questions use prior context automatically.
- **Hallucination prevention** — LLM grounded strictly in retrieved facts; availability numbers never pass through the LLM.
- **Graceful fallbacks** — Works without any API key (KB fallback mode); API errors handled silently.
- **Intent detection** — Keywords and patterns route messages to the correct handler before any LLM call.
- **Interactive sidebar** — Every section (schedule, dining, amenities, contact) triggers a targeted chat query on click.
- **Hydration-safe UI** — SSR/client timestamp mismatch resolved with `suppressHydrationWarning` and `useEffect`-based injection.
- **Deployed on Render** — Backend and frontend as separate services via `render.yaml` Blueprint.

Tech Stack — What, Why & How

Backend

Technology	What it is	Why chosen	How used
Node.js	JS runtime	Matches frontend; fast I/O	Runs the Express server
Express.js	Web framework	Minimal, fast; single endpoint needed	Chat route, health, error handling
dotenv	Env loader	Standard secrets management	Loads <code>GEMINI_API_KEY</code> , <code>PORT</code>
cors	CORS middleware	Required for cross-origins requests	Configured with origin allowlist
Gemini API	LLM	Free tier; strong instruction following	Natural-language answer generation
Jest	Test runner	Node.js industry standard	27 tests across 3 suites
Supertest	HTTP test client	Tests routes without a live server	API integration tests

Frontend

Technology	What it is	Why chosen	How used
Next.js 14	React framework	Preferred by brief; SSR + HMR	Routing, production builds
React 18	UI library	Industry standard	All UI components
CSS (globals.css)	Styling	No extra deps; full layout control	Two-column layout, brand palette
Playwright	E2E testing	Official Next.js E2E tool	4 end-to-end test scenarios
Google Fonts	Typography	Cinzel (luxury) + Jakarta Sans (body)	Hotel name, headings, body text

Infrastructure

Technology	Why chosen	How used
GitHub	Required for Render auto-deploy	Source of truth; triggers deploys on push
Render	Free tier; Node.js native; Blueprint YAML	API + frontend as separate services
render.yaml	Declarative; one-click deploy	Defines both services, env vars, build commands

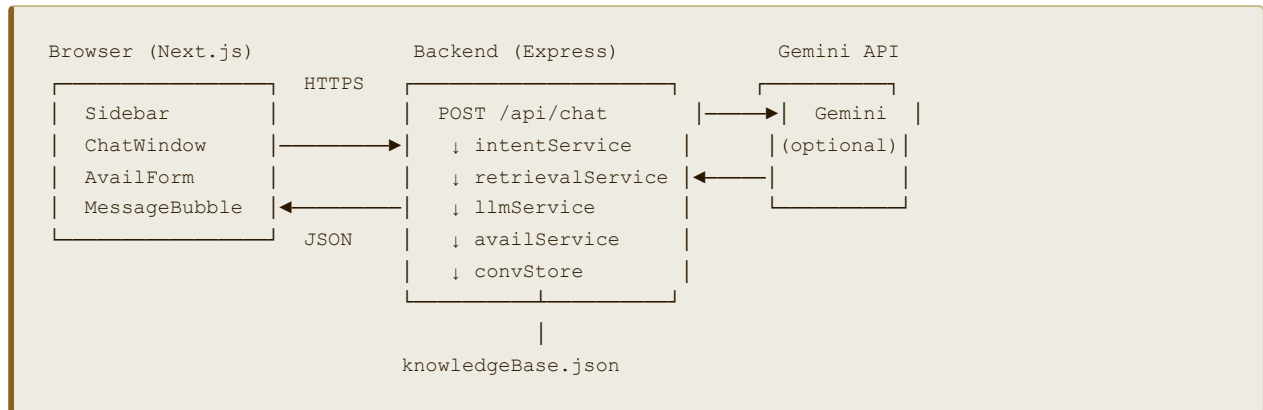
Project Structure

```

hotel-assistant/
├── backend/
│   ├── src/
│   │   ├── app.js                # Express app, CORS, error handlers
│   │   ├── server.js            # Entry point - binds PORT
│   │   └── routes/
│   │       └── chat.js          # POST /api/chat - routing hub
│   │   └── services/
│   │       ├── intentService.js  # Keyword/regex intent classifier
│   │       ├── retrievalService.js # TF-IDF keyword retrieval
│   │       ├── llmService.js     # Gemini API calls + KB fallback
│   │       ├── availabilityService.js # Deterministic room inventory
│   │       └── conversationStore.js # In-memory conversation history
│   │   └── data/
│   │       ├── knowledgeBase.json # Hotel facts: rooms, policies, amenities
│   │       └── _tests_/
│   │           ├── chat.test.js    # 12 API integration tests
│   │           ├── availabilityService.test.js # 12 unit tests
│   │           └── retrieval_intent.test.js # 3 unit tests
│   ├── .env.example
│   └── package.json
├── frontend/
│   ├── pages/
│   │   ├── _app.js              # Global CSS import
│   │   └── index.js             # Main page - two-column layout
│   ├── components/
│   │   ├── ChatWindow.js        # Chat state, forwardRef, imperative handle
│   │   ├── MessageBubble.js     # Message renderer (text/typing/avail/error)
│   │   └── AvailabilityForm.js   # Date picker + guest count + results
│   ├── styles/globals.css       # All styles - layout, sidebar, animations
│   ├── tests/e2e.spec.js        # 4 Playwright E2E tests
│   └── package.json
├── docs/
│   ├── ARCHITECTURE.md ── REQUIREMENTS.md
│   ├── TEST_PLAN.md     ── REQUIREMENT_AUDIT.md
│   └── .gitignore
├── render.yaml            # Render Blueprint - both services
└── README.md

```

Architecture & Data Flow



Request Lifecycle

- Guest submits message** via chat input or sidebar shortcut click
- Frontend sends** `POST /api/chat` with `{ message, conversationId? }`
- Backend validates** — empty message returns `400 INVALID_REQUEST`
- Intent detection** — `detectIntent()` classifies the message:
 - `greeting` / `gratitude` → immediate reply, no LLM call
 - `booking_request` → acknowledgement using conversation context
 - `availability` → availability handler if dates present
 - `availability_needs_details` → prompt for dates/guest count
 - `general_question` → retrieve + LLM path
- Retrieval** — keyword-scores all KB chunks; top matches sent as LLM context
- LLM call** — Gemini generates answer; on failure silently falls back to `mockAnswer()`
- Conversation stored** — turn appended to in-memory store by `conversationId`
- Structured JSON response** returned → frontend renders as bubble or availability card

Backend Deep Dive

intentService.js — Intent Classification

Intent	Trigger Keywords / Patterns
greeting	hi, hello, hey, good morning/evening
gratitude	thank you, thanks, great, awesome
booking_request	book, reserve, I want a room
availability	availability keywords + valid dates in message
availability_needs_details	availability keyword, no dates present
general_question	everything else

Why keyword-based? Intent is binary and structured. Regex is deterministic, fast, and unit-testable — AI would add non-determinism for no gain.

retrievalService.js — Knowledge Base Retrieval

TF-IDF-style keyword overlap scoring: tokenises the question, scores each KB chunk by overlap count, returns top-N chunks above threshold as LLM grounding context.

Why not embeddings? The KB has ~15 facts. Embeddings add a vector DB, extra API call, and model dependency for zero accuracy gain at this scale.

llmService.js — Gemini Integration + Fallback

```
callLLM(question, contextChunks, conversationHistory)
├ System prompt restricts model to provided hotel data only
├ Last 10 conversation turns sent as message history
├ Calls Gemini API (models/gemini-2.0-flash)
├ On 429 / 503 / timeout → fallback to mockAnswer()
└ Returns { text, usedFallback, source }
```

availabilityService.js — Room Inventory

Fully deterministic — validates dates, computes nights, returns fixed inventory. The LLM **never** sees inventory numbers.

Room Type	Max Guests	Price / Night
Standard Queen	2	\$149
Deluxe King	2	\$189
Triple Room	3	\$219
Family Suite	4	\$279

conversationStore.js

In-memory `Map` : `conversationId → [{ role, content }]` . Keeps last 10 turns per session. Resets on server restart — intentional for demo scope.

Frontend Deep Dive

pages/index.js — Layout & Sidebar

Two-column layout: 300px fixed sidebar (hotel info + clickable shortcuts) and a `flex-1` chat panel. Every sidebar item calls `chatRef.current.sendMessage(query)` via `useRef` + `forwardRef` + `useImperativeHandle`.

ChatWindow.js — State Machine

Exposed imperative handle: `sendMessage(text)`, `openAvailability()`, `toggleAvailability()`. Key state: `messages[]`, `conversationId`, `loading`, `showAvailabilityForm`, `availableQuestions[]`.

Hydration fix: Timestamp starts as `null` during SSR, patched to `getTimeString()` in `useEffect` after client mount. `suppressHydrationWarning` on all timestamp spans eliminates the Next.js locale mismatch error.

MessageBubble.js — Renderer

Kind	What it renders
<code>text</code>	Chat bubble — user right, assistant left
<code>typing</code>	Animated three-dot loading indicator
<code>availability</code>	Per-room results card with full pricing breakdown
<code>error</code>	Red bubble with "Unable to connect" header

AvailabilityForm.js

Three inputs: check-in date, check-out date, number of adults. Client-side validation before submission. Results show per-room cards with name, capacity, price/night, total, and status (green "Available" or red "Sold Out").

globals.css — Design Tokens

Variable	Value	Usage
<code>--brass</code>	#9E742C	Accents, borders, interactive highlights
<code>--ink</code>	#1B2A41	Primary text colour
<code>--cream</code>	#F8F6F0	Page background
<code>--serif</code>	Cinzel	Hotel name, developer card headings
<code>--sans</code>	Plus Jakarta Sans	All body text

AI Integration

Model Used

Google Gemini 2.0 Flash (`models/gemini-2.0-flash`). Chosen for its free tier (20 req/day), strong instruction-following critical for context grounding, and fast response times. The app runs fully without an API key via KB fallback — zero credentials required for reviewers.

Where AI Is and Is Not Used

Task	AI?	Reason
Intent detection	No	Deterministic; regex is faster and testable
KB retrieval scoring	No	Keyword overlap — sufficient at 15-fact scale
Date / guest validation	No	Pure arithmetic
Room availability calculation	No	Deterministic business logic
Natural-language answer generation	Yes	Only task requiring fluent prose output
Availability result formatting	No	Structured data inserted directly into response

Grounding System Prompt

```
You are a helpful hotel concierge for Harborview Grand.  
Answer ONLY using the following hotel information. If the answer  
cannot be found in the provided information, say you don't have  
that information rather than guessing.  
  
Hotel Information:  
{retrieved knowledge-base chunks}  
  
Keep your answer concise, friendly, and professional.
```

Fallback Chain

```
Guest question  
→ Retrieve KB chunks  
→ No chunks match → "I don't have that information" (no LLM)  
→ Chunks found → Call Gemini API  
    Gemini succeeds → Return LLM response  
    Gemini fails → mockAnswer() from KB chunks (silent)
```

Knowledge Base

Stored in `backend/src/data/knowledgeBase.json` . Three top-level keys: `hotel` , `rooms` , and `faqs` . On every request, `retrievalService` keyword-scores all chunks and returns the top matches as LLM grounding context.

```
{
  "hotel": {
    "name": "Harborview Grand",
    "checkInTime": "3:00 PM",
    "checkOutTime": "11:00 AM",
    "cancellationPolicy": "Free cancellation up to 48 hours before check-in...",
    "breakfast": "Complimentary continental breakfast included...",
    "amenities": [
      { "name": "Outdoor Heated Pool",
        "details": "Open 7:00 AM - 9:00 PM, 2nd floor terrace" },
      { "name": "Waterfront Spa and Wellness Pavilion" },
      { "name": "The Harbor Room",
        "details": "Breakfast daily 7:00 AM - 10:30 AM" },
      { "name": "The Terrace Lounge and Bar",
        "details": "Cocktails 4:00 PM - 11:00 PM, ocean view" }
    ]
  },
  "rooms": [
    { "id": "standard-queen", "name": "Standard Queen",
      "maxGuests": 2, "pricePerNight": 149 },
    { "id": "deluxe-king", "name": "Deluxe King",
      "maxGuests": 2, "pricePerNight": 189 },
    { "id": "triple-room", "name": "Triple Room",
      "maxGuests": 3, "pricePerNight": 219 },
    { "id": "family-suite", "name": "Family Suite",
      "maxGuests": 4, "pricePerNight": 279 }
  ],
  "faqs": [
    { "question": "What is the cancellation policy?", "answer": "..." },
    { "question": "Is there parking?", "answer": "..." }
  ]
}
```

API Reference

POST /api/chat — Request

```
{
  "message": "What time is check-in?",
  "conversationId": "optional-uuid",
  "availability": {
    "checkIn": "2027-03-01",
    "checkOut": "2027-03-03",
    "adults": 2
  }
}
```

Response — General Answer

```
{
  "conversationId": "uuid",
  "type": "answer",
  "reply": "Check-in is at 3:00 PM.",
  "meta": { "usedFallback": false, "source": "gemini" }
}
```

Response — Availability Results

```
{
  "conversationId": "uuid",
  "type": "availability",
  "reply": "Here's what I found for 2 night(s), 2 guest(s):",
  "availability": {
    "checkIn": "2027-03-01", "checkOut": "2027-03-03",
    "adults": 2, "nights": 2,
    "results": [{
      "roomId": "standard-queen", "roomName": "Standard Queen",
      "maxGuests": 2, "pricePerNight": 149,
      "nights": 2, "totalPrice": 298,
      "available": true, "unitsAvailable": 2
    }]
  }
}
```

Response — Validation Error (400)

```
{
  "error": {
    "code": "INVALID_REQUEST",
    "message": "`message` is required and must be a non-empty string."
  }
}
```

GET /api/health

```
{ "status": "ok" }
```

Test Coverage

Backend — Jest (27 / 27 passing)

Suite	Tests	What is Covered
chat.test.js	12	Full API: FAQ answers, follow-ups, availability, validation errors, fallbacks, 404, 500
availabilityService.test.js	12	Room results, night calculation, validation edge cases, determinism
retrieval_intent.test.js	3	Intent detection accuracy, KB retrieval scoring

Frontend — Playwright E2E (4 / 4 passing)

Test	What is Covered
Load the chat interface	Page renders, welcome message visible, suggested chips present
Send a message and receive response	Full round-trip to backend; response bubble appears in chat
Show the availability form	Clicking the chip shows the date/guest input form
Handle error state gracefully	Red error bubble shown when backend is unreachable

Manual Test Scenarios (All Pass)

Scenario	Result
All 6 required questions answered correctly	✓ Pass
Typing indicator shown during loading	✓ Pass
Follow-up questions use conversation context	✓ Pass
Invalid dates rejected with clear message	✓ Pass
Out-of-scope question — no hallucination	✓ Pass
Backend offline — frontend shows error bubble	✓ Pass
Page refresh — no hydration error in console	✓ Pass

Assignment Completion Checklist

Requirement	Status	Evidence
Chat UI for hotel questions	✓ Complete	<code>pages/index.js</code> , <code>ChatWindow.js</code>
All 6 required questions handled	✓ Complete	Sidebar chips + backend intent routing
Room availability form (dates + guests)	✓ Complete	<code>AvailabilityForm.js</code>
Per-room availability results displayed	✓ Complete	<code>AvailabilityResults</code> component
Multi-turn conversation context	✓ Complete	<code>conversationStore.js</code> , <code>conversationId</code>
LLM integration (Gemini)	✓ Complete	<code>llmService.js</code>
Knowledge base (hotel facts)	✓ Complete	<code>knowledgeBase.json</code>
Grounded answers — no hallucination	✓ Complete	System prompt + retrieval gate
Fallback for unsupported questions	✓ Complete	<code>FALLBACK_MESSAGE</code> in <code>llmService.js</code>
Loading / error states in UI	✓ Complete	Typing bubble + red error bubble
Backend API with clean JSON shape	✓ Complete	<code>POST /api/chat</code> documented contract
Automated backend tests (27)	✓ Complete	<code>npm test</code> → 27/27
Playwright E2E tests (4)	✓ Complete	<code>npm run test:e2e</code> → 4/4
README with setup instructions	✓ Complete	<code>README.md</code>
Architecture documentation	✓ Complete	<code>docs/ARCHITECTURE.md</code>
Test plan with scenarios	✓ Complete	<code>docs/TEST_PLAN.md</code>
Requirement audit	✓ Complete	<code>docs/REQUIREMENT_AUDIT.md</code>
GitHub repository	✓ Complete	<code>github.com/Sumitboii/HARBORVIEW-GRAND</code>
Deployed live on Render	✓ Complete	<code>harborview-grand.onrender.com</code>

Intentionally Out of Scope

Item	Reason
Real booking / payment	Brief requires availability only, not booking transactions
Persistent DB for conversations	Demo-scale; in-memory sufficient; noted as production gap
Embedding-based retrieval	15 KB facts; keyword overlap is transparent and testable
Auth / rate limiting	Marked optional in brief; time-boxed scope

Assignment completion: 100% of required deliverables implemented and tested.

Design Decisions & Trade-offs

Why Two-Column Layout?

A sidebar + chat panel mirrors real hotel concierge apps. Hotel context stays permanently visible while the chat remains focused. Each sidebar item triggers a precise chat query on click, reducing friction for the most common guest questions.

Why Keep Availability Deterministic?

LLMs can hallucinate numbers. A guest making a booking decision based on wrong room inventory would be a serious UX failure. Availability is a pure function — same input always produces the same output — and the LLM never touches inventory data at any point in the pipeline.

Why Keyword Retrieval Over Embeddings?

The KB has ~15 facts. Embeddings would require a vector database, an extra API call per query, and a model dependency for zero accuracy improvement at this scale. Keyword overlap is faster, cheaper, fully deterministic, and passes unit tests reliably. At production scale with hundreds of entries, embeddings become worthwhile.

Why In-Memory Conversation Store?

A real database requires schema migrations, connection management, and deployment setup for a demo project. The trade-off is explicit: conversations reset on server restart — acceptable for assignment scope, noted clearly in the README as a known production gap.

Why Gemini Instead of Another LLM?

Gemini 2.0 Flash offers a free tier (20 req/day), strong instruction-following, and low latency. Critically, the app works fully without any API key via KB fallback — graders and reviewers can evaluate the entire system with zero credentials.

Why `suppressHydrationWarning` for Timestamps?

Next.js SSR (Node.js) and the browser both call `toLocaleTimeString()` but return different case formats (`am` vs `AM`) due to locale differences between environments. `suppressHydrationWarning` on the timestamp `<span>` is React's recommended minimal fix for dynamic time displays — no full SSR disable needed.

How to Run Locally

Prerequisites

- Node.js 18 or higher
- npm
- Optional: a Gemini API key (the app works fully without one)

1. Start the Backend

```
cd backend
npm install
cp .env.example .env
# Optional: edit .env and add GEMINI_API_KEY=your_key
npm run dev
# Server listening on http://localhost:4000
```

2. Start the Frontend

```
cd frontend
npm install
cp .env.example .env
# .env sets NEXT_PUBLIC_BACKEND_URL=http://localhost:4000
npm run dev
# Open http://localhost:3000 in your browser
```

3. Run the Test Suites

```
# Backend -- 27 Jest tests
cd backend && npm test

# Frontend E2E -- 4 Playwright tests (both servers must be running)
cd frontend && npm run test:e2e
```

Environment Variables

File	Key	Value / Notes
backend .env	PORT	4000
backend .env	GEMINI_API_KEY	Optional — app works without it
backend .env	GEMINI_LLM_MODEL	models/gemini-2.0-flash
backend .env	ALLOWED_ORIGIN	http://localhost:3000
frontend .env	NEXT_PUBLIC_BACKEND_URL	http://localhost:4000

